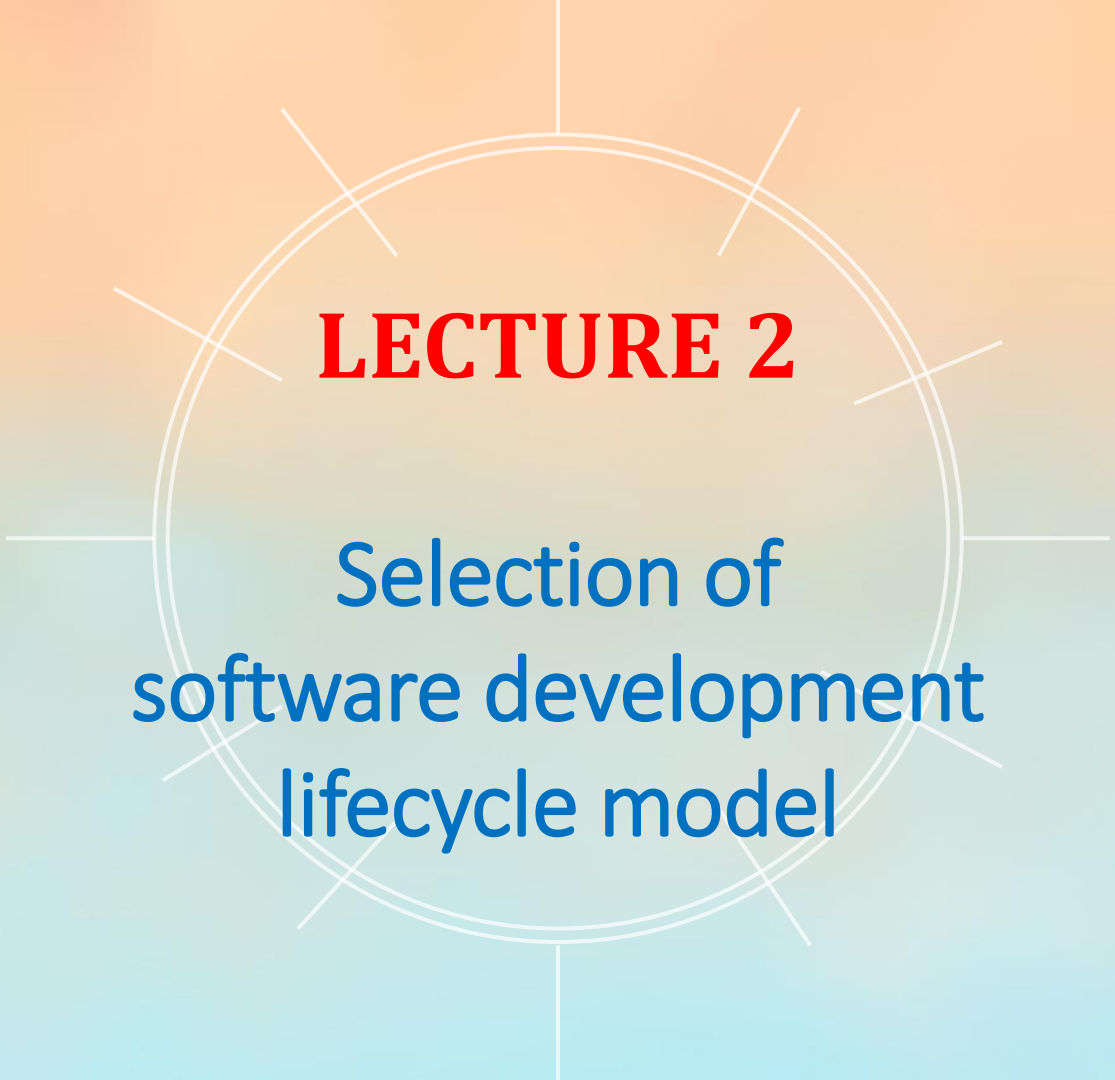


A decorative graphic consisting of a large circle with several thin, light-colored lines radiating from its center to the edge, resembling a compass or a stylized sun. The background is a gradient of light orange at the top and light blue at the bottom.

LECTURE 2

Selection of
software development
lifecycle model

Content

1

Software development lifecycle (SDLC) revision

2

Some typical SDLC models

3

Selecting appropriate SDLC model for software project

Software Development Lifecycle

- When software systems were first developed in the mid twentieth century they were done so in a rather ad-hoc manner without any reference to a defined development process.
- It was soon realized that this undisciplined approach led to systems that were poorly constructed and difficult to manage and control.
- Some form of development process was required that could assist software developers in producing more structured code in a more manageable way.

Software Development Lifecycle

- The advantages of structuring software development into a defined process model are that:
 - It divides a large problem down into easy-to-understand tasks at each stage.
 - It sharpens focus.
 - It supports planning and control – improves time estimates.
 - It provides progress visibility.
 - It provides structure.
 - It leads to better coding and documentation.

Software Development Lifecycle

- The Software Development Life-Cycle (SDLC) represents a generic model for software development and consists of a number of stages including requirements analysis, design, development, testing and implementation.

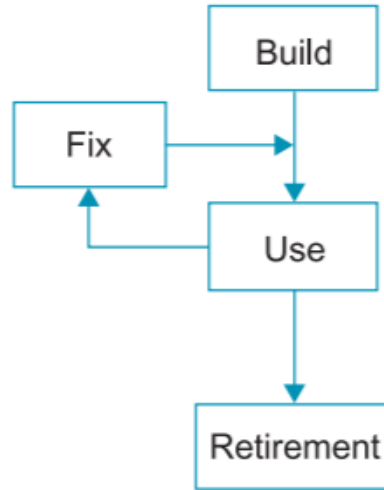


The 'Build-and-Fix' Model

- In the pioneering days of software development there was no recognized process for developing software.
- The build-and-fix or code-and-fix 'model' represents this early approach to the development of software.
- There is no formal requirements capture in the process and no formal design. Programmers would first write some code, run the code and then correct any bugs in the software.
- In this case there is no formal breakdown of the process into stages.

The 'Build-and-Fix' Model

- The model merely iterates until the software becomes unworkable and is eventually retired and/or replaced.



The build-and-fix 'model'

The 'Build-and-Fix' Model

- Disadvantages:
 - ✓ After several fixes the software becomes difficult to maintain as it becomes poorly structured.
 - ✓ It often does not match the user's requirements – it is rejected or requires extensive redevelopment.
 - ✓ It can be costly to maintain because of its poor structure and lack of definable output that can be tested.
- Although the build-and-fix approach is still used today by many programmers working on small and personal systems, you should not be using it for your project.
- You should, instead, look to one of the more defined and recognized processes.

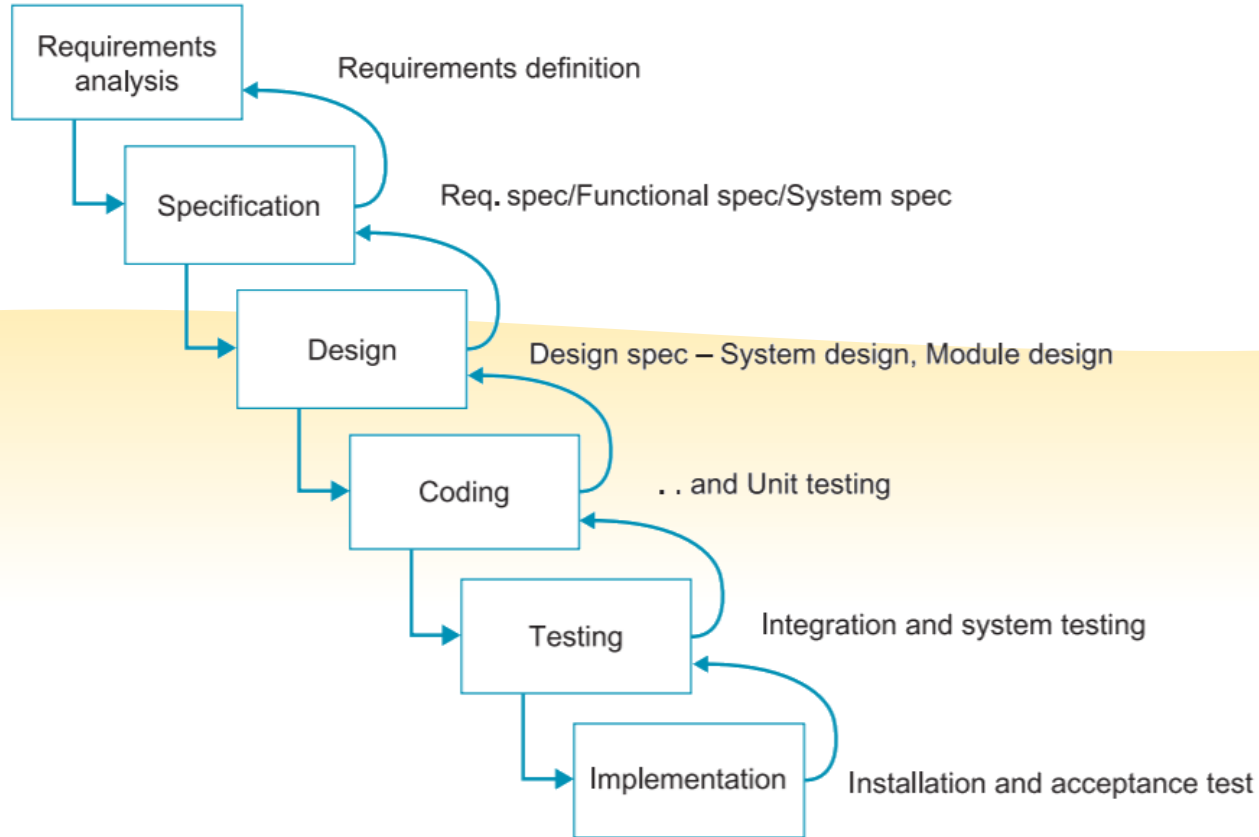
The Waterfall Model

- Owing to the problems encountered with the unstructured approach of the build-and-fix model, several more detailed models were devised.
- The earliest of these models was the stage-wise model from which the waterfall model developed.
- The stage-wise model was developed in 1956 by Benington in an attempt to provide an engineering process to the development of software.
- It represents a unidirectional, sequential process – once a stage has been completed, the results of that stage become a fixed baseline from which the following stages develop – there is no revision.

The Waterfall Model

- However, that the sequential nature of the stage-wise model was causing problems and some form of reworking was required to allow a user's needs to be addressed more effectively. For this reason, the Waterfall model was developed.
- The Waterfall model differed from the stage-wise model in that it allowed some limited iteration between stages – shown as 'splashing back'

The Waterfall Model



The Incremental Model

- Rather than delivering your software to your client towards the end of your project as the conventional models do, it might be better to deliver the system to them as a series of intermediate working subsystems over a period of time.
- Thus, you add more functionality to the software at each release of the system. This means that you need to get an overall software structure (kernel) in place as part of the first release of your system.
- The other parts of the system are then brought online and released to the user as the system is developed. Thus, each release to the user provides added functionality to the existing system.